

[Algorithm]

집합 커버(SetCover) 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



집합 커버 문제

□ 문제

- n 개의 원소를 가진 집합 U 가 있고,
- U 의 부분집합들을 원소로 하는 집합 F 가 주어질 때,
- F 의 원소들인 집합들 중에서 어떤 집합들을 선택하여 합집합하면 U 와 같게 되는가?

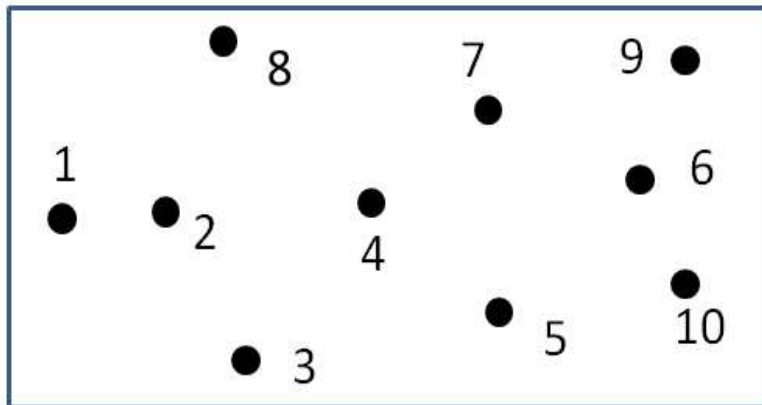
□ 집합 커버 (Set Cover) 문제

- 집합 F 에서 선택하는 집합들의 수를 최소화하는 문제

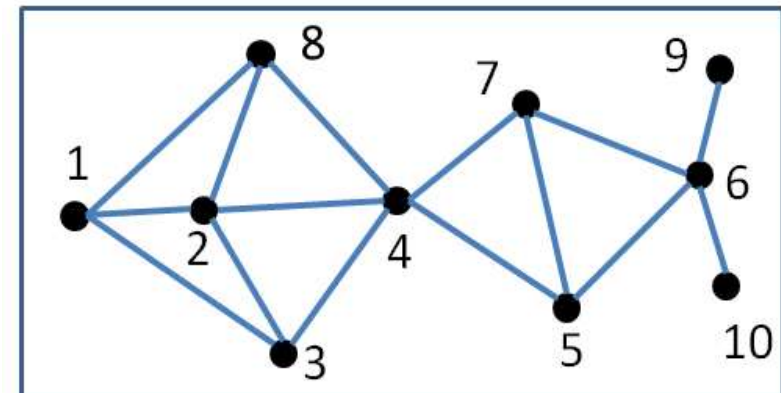
신도시 학교 배치 문제 예

□ 신도시를 계획하는 데 있어서 학교 배치의 예

- 10개의 마을이 신도시에 만들어질 계획이다.
- 아래의 2가지 조건이 만족되도록 학교 위치를 선정해야 한다.
 - 학교는 마을에 위치해야 한다.
 - 등교 거리는 걸어서 15분 이내이어야 한다.
- 어느 마을에 학교를 신설해야 학교의 수가 최소가 되는가?



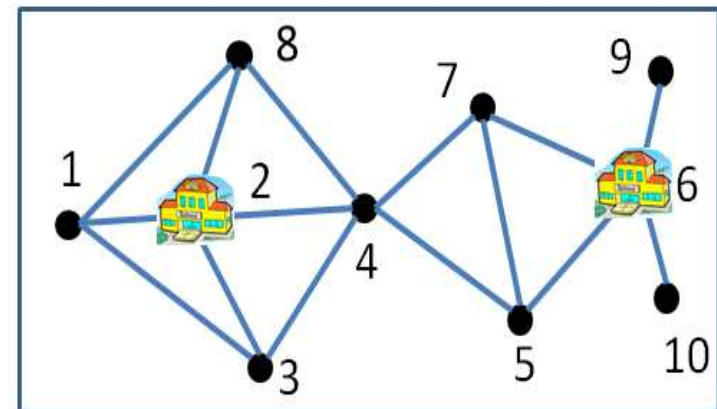
10개 마을의 위치



등교 거리가 15분 이내인 마을 간의 관계

신도시 학교 배치 문제 예

- 어느 마을에 학교를 신설해야 학교의 수가 최소가 되는가?
 - 2번 마을에 학교를 만들면
 - 1, 2, 3, 4, 8 마을의 학생들이 15분 이내에 등교 가능
 - 즉, 마을 1, 2, 3, 4, 8이 커버된다.
 - 6번 마을에 학교를 만들면
 - 마을 5, 6, 7, 9, 10이 커버된다.
 - 2번과 6번 마을에 학교를 배치하면 모든 마을이 커버된다.
 - 최소의 학교 수는 2개



신도시 학교 배치 문제 예

□ 신도시 계획 문제를 집합 커버 문제로 변환

$U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ // 신도시의 마을 10개

$F = \{S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}\}$

// S_i 는 마을 i 에 학교를 배치했을 때 커버되는 마을의 집합

$S_1 = \{1, 2, 3, 8\}$

$S_5 = \{4, 5, 6, 7\}$

$S_9 = \{6, 9\}$

$S_2 = \{1, 2, 3, 4, 8\}$

$S_6 = \{5, 6, 7, 9, 10\}$

$S_{10} = \{6, 10\}$

$S_3 = \{1, 2, 3, 4\}$

$S_7 = \{4, 5, 6, 7\}$

$S_4 = \{2, 3, 4, 5, 7, 8\}$

$S_8 = \{1, 2, 4, 8\}$

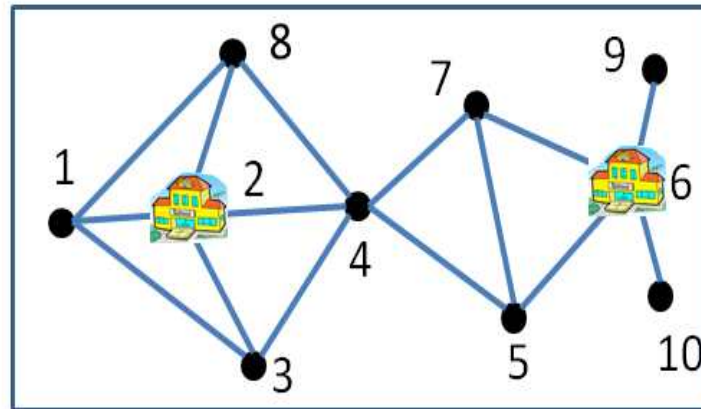
– S_i 집합들 중에서 어떤 집합들을 선택해야 그들의 합집합이 U 와 같은가?

- 단, 선택된 집합의 수는 최소이어야 한다.

신도시 학교 배치 문제 예

□ 문제의 답

$$\begin{aligned} - S_2 \cup S_6 &= \{1, 2, 3, 4, 8\} \cup \{5, 6, 7, 9, 10\} \\ &= \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} = U \end{aligned}$$



집합 커버 문제의 최적해

□ 집합 커버 문제의 최적해는 어떻게 찾아야 할까?

- F에 n개의 집합들이 있다고 가정해보자.

□ 가장 단순한 방법

- F에 있는 집합들의 모든 조합을 1개씩 합집합하여 U가 되는지 확인하고,
- U가 되는 조합의 집합 수가 최소인 것을 찾는 것이다.

- $F = \{S_1, S_2, S_3\}$ 일 경우 모든 조합

- $S_1, S_2, S_3, S_1 \cup S_2, S_1 \cup S_3, S_2 \cup S_3, S_1 \cup S_2 \cup S_3$
- 집합이 1개인 경우 3개 = ${}_3C_1$
- 집합이 2개인 경우 3개 = ${}_3C_2$
- 집합이 3개인 경우 1개 = ${}_3C_3$
- 총합은 $3+3+1=7=2^3-1$ 개

집합 커버 문제의 최적해

□ n 개의 원소가 있을 경우

- $(2^n - 1)$ 개를 다 검사하여야 한다.
- n 이 커지면 최적해를 찾는 것은 실질적으로 불가능

□ 이를 극복하기 위한 방법

- 최적해를 찾는 대신에 최적해에 근접한 근사해 (approximation solution)를 찾는다.
- 정답과 거의 비슷한 답 유추
- 평가
 - 얼마나 빠르냐 & 최적해에 얼마나 가까운가

집합 커버 알고리즘

SetCover

입력: U , $F=\{S_i\}$, $i=1,\dots,n$

출력: 집합 커버 C

1. $C=\emptyset$
2. while ($U\neq\emptyset$) do {
3. U 의 원소들을 가장 많이 포함하고 있는 집합 S_i 를 F 에서 선택한다.
4. $U=U-S_i$
5. S_i 를 F 에서 제거하고, S_i 를 C 에 추가한다.
- }
6. return C

집합 커버 알고리즘

□ Line 1

- C 를 공집합으로 초기화 시킨다.

□ Line 2~5

- while-루프에서는 집합 U 가 공집합이 될 때까지 수행된다.

□ Line 3

- 그리디하게 U 와 가장 많은 수의 원소들을 공유하는 집합 S_i 를 선택한다.

집합 커버 알고리즘

□ Line 4

- S_i 의 원소들을 U 에서 제거한다.
- 왜냐하면 S_i 의 원소들은 커버된 것이기 때문이다.
- 따라서 U 는 아직 커버되지 않은 원소들의 집합이다.

□ Line 5

- S_i 를 F 로부터 제거하여, S_i 가 line 3에서 더 이상 고려되지 않도록 하며, S_i 를 집합 커버 C 에 추가한다.

□ Line 6

- C 를 리턴한다.

SetCover 알고리즘의 수행 과정

$$U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

$$F = \{S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}\}$$

$$S_1 = \{1, 2, 3, 8\}$$

$$S_6 = \{5, 6, 7, 9, 10\}$$

$$S_2 = \{1, 2, 3, 4, 8\}$$

$$S_7 = \{4, 5, 6, 7\}$$

$$S_3 = \{1, 2, 3, 4\}$$

$$S_8 = \{1, 2, 4, 8\}$$

$$S_4 = \{2, 3, 4, 5, 7, 8\}$$

$$S_9 = \{6, 9\}$$

$$S_5 = \{4, 5, 6, 7\}$$

$$S_{10} = \{6, 10\}$$

SetCover 알고리즘의 수행 과정

□ Line 1

- $C = \emptyset$ 로 초기화

□ Line 2

- while-조건 ($U \neq \emptyset$) = ($\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} \neq \emptyset$)이 '참'이다.

□ Line 3

- U 의 원소를 가장 많이 커버하는 집합인 $S_4 = \{2, 3, 4, 5, 7, 8\}$ 을 F 에서 선택한다.

□ Line 4

- $U = U - S_4 = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} - \{2, 3, 4, 5, 7, 8\} = \{1, 6, 9, 10\}$

□ Line 5

- S_4 를 F 에서 제거하고, 즉, $F = \{S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}\} - \{S_4\} = \{S_1, S_2, S_3, S_5, S_6, S_7, S_8, S_9, S_{10}\}$ 가 되고,
- S_4 를 C 에 추가한다. 즉, $C = \{S_4\}$ 이다.

SetCover 알고리즘의 수행 과정

□ Line 2

- while-조건 ($U \neq \emptyset$) = ($\{1, 6, 9, 10\} \neq \emptyset$)이 '참'이다.

□ Line 3

- U 의 원소를 가장 많이 커버하는 집합인 $S_6 = \{5, 6, 7, 9, 10\}$ 을 F 에서 선택한다.

□ Line 4

- $U = U - S_4 = \{1, 6, 9, 10\} - \{5, 6, 7, 9, 10\} = \{1\}$

□ Line 5

- S_6 을 F 에서 제거하고, 즉, $F = \{S_1, S_2, S_3, S_5, S_6, S_7, S_8, S_9, S_{10}\} - \{S_6\} = \{S_1, S_2, S_3, S_5, S_7, S_8, S_9, S_{10}\}$ 이 되고,
- S_6 을 C 에 추가한다. 즉, $C = \{S_4, S_6\}$ 이다.

SetCover 알고리즘의 수행 과정

□ Line 2

- while-조건 ($U \neq \emptyset$) = ($\{1\} \neq \emptyset$)이 '참'이다.

□ Line 3

- U 의 원소를 가장 많이 커버하는 집합인 $S_1 = \{1, 2, 3, 8\}$ 을 F 에서 선택한다.
- S_1 대신에 S_2, S_3, S_8 중에서 어느 하나를 선택해도 무방하다.

□ Line 4

- $U = U - S_1 = \{1\} - \{1, 2, 3, 8\} = \emptyset$

□ Line 5

- S_1 을 F 에서 제거하고, 즉, $F = \{S_1, S_2, S_3, S_5, S_6, S_7, S_8, S_9, S_{10}\} - \{S_1\} = \{S_2, S_3, S_5, S_7, S_8, S_9, S_{10}\}$ 이 되고,
- S_1 을 C 에 추가한다. 즉, $C = \{S_1, S_4, S_6\}$ 이다.

SetCover 알고리즘의 수행 과정

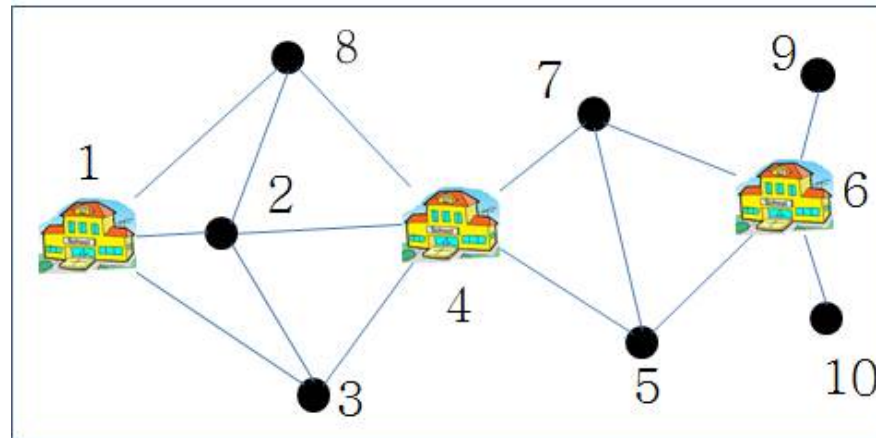
□ Line 2

- while-조건 ($U \neq \emptyset$) = ($\emptyset \neq \emptyset$)이 '거짓'이므로, 루프를 끝낸다.

□ Line 6

- $C = \{S_1, S_4, S_6\}$ 을 리턴한다.

□ SetCover 알고리즘의 최종해



시간복잡도

- 먼저 while-루프가 수행되는 횟수는 최대 n 번이다.
 - 왜냐하면 루프가 1번 수행될 때마다 집합 U 의 원소 1개씩만 커버된다면, 최악의 경우 루프가 n 번 수행되어야 하기 때문이다.

시간복잡도

□ 루프가 1번 수행될 때의 시간복잡도

- Line 2의 while-루프 조건 ($U \neq \emptyset$)을 검사하는 $O(1)$ 시간이 걸린다.
 - 왜냐하면 U 의 현재 원소 수를 위한 변수를 두고 그 값이 0인지를 검사하면 되기 때문이다.
- Line 3: U 의 원소들을 가장 많이 포함하고 있는 집합 S 를 찾으려면, 현재 남아있는 S_i 들 각각을 U 와 비교하여야 한다.
- 따라서 S_i 들의 수가 최대 n 이라면, 각 S_i 와 U 의 비교는 $O(n)$ 시간이 걸리므로, line 3은 $O(n)$ 시간이 걸린다.
- Line 4: 집합 U 에서 집합 S_i 의 원소를 제거하는 것이므로 $O(n)$ 시간이 걸린다.
- Line 5: S_i 를 F 에서 제거하고, S_i 를 C 에 추가하는 것이므로 $O(1)$ 시간이 걸린다.

시간복잡도

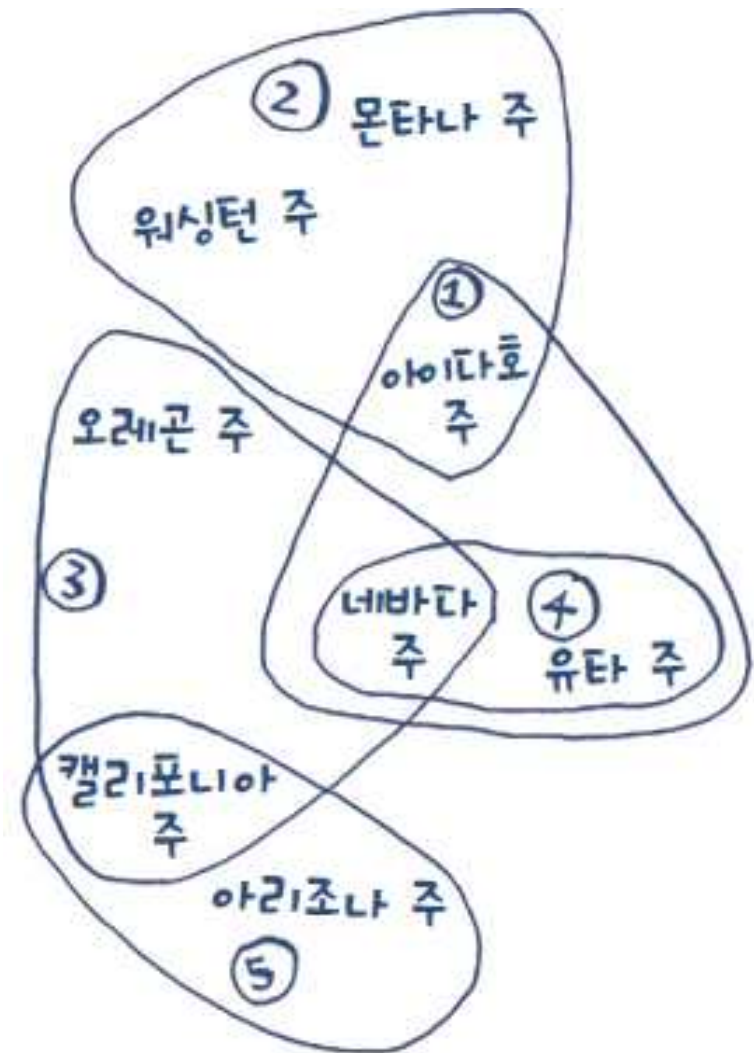
- 루프가 수행되는 횟수
 - $O(n)$
- 루프 1회의 시간복잡도
 - $O(1)+O(n)+O(n)+O(1) = O(n)$
- SetCover 알고리즘의 시간복잡도
 - $O(n) \times O(n) = O(n^2)$ 이다.

방송국 선택 문제 예

□ Ex) 라디오 쇼를 최대한 적은 수의 방송국을 통해 미국 50개 주 모두에 송출하려면?

라디오 방송국	청취 가능한 주
KONE	아이다호 주, 네바다 주, 유타 주
KTWO	워싱턴 주, 아이다호 주, 몬타나 주
KTHREE	오레곤 주, 네바다 주, 캘리포니아 주
KFOUR	네바다 주, 유타 주
KFIVE	캘리포니아 주, 아리조나 주

기타 등등



방송국 선택 문제 예

□ 첫번째 방법

- 가능한 모든 방송국의 부분 집합을 나열
 - $2^N - 1$ 개
- 그 중 50개 주 전체를 커버하면서 가장 원소 수가 작은 집합 선택
- $O(2^N)$ 시간 소요
 - 항목 수가 증가한다면 매우 비효율적

방송국의 수	전체 실행 시간
5	3.2초
10	102.4초
32	13.6년
100	4×10^{21} 년

□ 두번째 방법(근사알고리즘)

- 아직 방송하지 않은 지역 중 가장 많은 지역에 방송 가능한 방송국 선택
- 모든 주에 방송이 될 때까지 선택 반복

방송국 선택 문제 예

- 코드로 풀 경우

- 준비 코드

- 방송하고자 하는 주의 목록 만들
» 집합 타입 사용

```
states_needed = set(["mt", "wa", "or", "id", "nv", "ut", "ca", "az"])
```

- 선택된 방송국의 목록

```
stations = {}  
stations["kone"] = set(["id", "nv", "ut"])  
stations["ktwo"] = set(["wa", "id", "mt"])  
stations["kthree"] = set(["or", "nv", "ca"])  
stations["kfour"] = set(["nv", "ut"])  
stations["kfive"] = set(["ca", "az"])
```

- 방문할 방송국 목록 저장할 집합

```
final_stations = set()
```

방송국 선택 문제 예

■ 답 계산

- 아직 방송 송출되지 않는 주 중 가장 많은 수 커버하는 방송국 선택

```
best_station = None
states_covered = set()
for station, states_for_station in stations.items():
```

- for 반복문 사용하여 어떤 것이 최선의 선택지인지 확인

```
    covered = states_needed & states_for_station
    if len(covered) > len(states_covered):
        best_station = station
        states_covered = covered
```

– covered = states_needed & states_for_station

» 아직 방송하지 않는 주 중 현재 고려하는 방송국이 커버하는 곳

» 이 집합이 best_station 보다 많은 주를 커버하는지 확인

» 그 경우 그 방송국을 새로운 best_station 으로

방송국 선택 문제 예

- for 반복문 끝나면 best_station 을 방송국 목록에 추가

```
final_stations.add(best_station)
```

- states_needed 갱신

```
states_needed -= states_covered
```

■ 전체 코드

```
while states_needed:
    best_station = None
    states_covered = set()
    for station, states in stations.items():
        covered = states_needed & states
        if len(covered) > len(states_covered):
            best_station = station
            states_covered = covered

    states_needed -= states_covered
    final_stations.add(best_station)
```


방송국 선택 문제 예

- 출력 결과

```
>>> print final_stations  
set(['ktwo', 'kthree', 'kone', 'kfive'])
```

- 실행 시간 비교

방송국의 수	$O(2^N)$ 정확한 알고리즘	$O(n^2)$ 탐욕 알고리즘
5	3.2 초	2.5 초
10	102.4 초	10 초
32	13.6 년	102.4 초
100	4×10^{21} 년	16.67 분

방송국 선택 문제 구현

```
states_needed = set(["mt", "wa", "or", "id", "nv", "ut", "ca", "az"])

stations = {}
stations["kone"] = set(["id", "nv", "ut"])
stations["ktwo"] = set(["wa", "id", "mt"])
stations["kthree"] = set(["or", "nv", "ca"])
stations["kfour"] = set(["nv", "ut"])
stations["kfive"] = set(["ca", "az"])

final_stations = set()

while states_needed:
    best_station = None
    states_covered = set()
    for station, states in stations.items():
        covered = states_needed & states
        if len(covered) > len(states_covered):
            best_station = station
            states_covered = covered

    states_needed -= states_covered
    final_stations.add(best_station)

print(final_stations)
```

응용

- 도시 계획 (City Planning)에서 공공 기관 배치하기
- 경비 시스템: 경비가 요구되는 장소의 CCTV 카메라의 최적 배치
- 컴퓨터 바이러스 찾기
- 대기업의 구매 업체 선정
- 기업의 경력 직원 고용
- 그 외에도 비행기 조종사 스케줄링, 조립 라인 균형화, 정보 검색 등에 활용



Thank You